الاحتراف الكامل

في مكتبات بايثون لعلم البيانات والتعلّم الآلي

الجزء الحادي عشر: Jupyter Notebook

الفصول ١ - ٦: من التعريف بالبيئة حتى المفاهيم الأساسية

مرجع احترافي للمبرمجين والمهندسين وعلماء البيانات

جدول المحتويات

جدول المحتويات 2
مقدمة 4
الفصل الأول: التعريف بـ Jupyter 5
١.١ النشأة والتطور 5
١.٢ المشكلة التي حلّها 5
١.٣ الاستخدامات والشركات 5
١.٤ المميزات والعيوب 5
١.٥ متى تستخدمه ومتى لا 5
١.٦ أشهر البدائل والامتدادات 6
ملخص الفصل 6
الفصل الثاني: التثبيت والإعداد 7
٢.١ التشغيل 7
٢.٢ إدارة Kernels متعددة البيئات 7
٢.٣ تنظيم المشروع 7
أفضل الممارسات 7
الفصل الثالث: البنية الداخلية ونموذج التنفيذ 8
٣.١ العمارة: Frontend وKernel منفصلان 8
٣.٢ Execution Model: حالة مشتركة عبر الخلايا 8
٣.٣ لماذا هذا يسبب مشاكل: مشكلة الحالة الخفية 8
٣.٤ أنواع الخلايا 8
ملخص الفصل 9
الفصل الرابع: المفاهيم الأساسية 10
٤.١ دورة حياة Kernel 10
٤.٢ Markdown داخل الدفاتر 10
٤.٣ Magic Commands 10
٤.٤ Widgets للتفاعلية 10
الأخطاء الشائعة 11
تمارين عملية 11
حلول مقترحة (تلميحات) 11
الفصل الخامس: الامتدادات وأدوات التصحيح والقياس 12
٥.١ التصحيح (Debugging) داخل Jupyter 12
٥.٢ القياس والملفات الشخصية (Profiling) 12
٥.٣ اختصارات لوحة المفاتيح الأساسية 12

٥.٤ الامتدادات الشائعة (Extensions) 12
أفضل الممارسات 12
الفصل السادس: التنظيم وأسلوب العمل الاحترافي 14
٦.١ هيكلة دفتر واحد جيدة التنظيم 14
٦.٢ من الاستكشاف إلى الإنتاج 14
٦.٣ عرض تقديمي (Presentation Mode) 14
٦.٤ التصدير والمشاركة 14
الأخطاء الشائعة 14
الأسئلة الشائعة 14
تمارين عملية 15
حلول مقترحة (تلميحات) 15
مراجع وروابط رسمية 15
خاتمة الجزء الأول من Jupyter 15

مقدمة

Jupyter Notebook مختلفة جوهريًا عن المكتبات الخمس السابقة: فهي ليست مكتبة تُستورد وتُستخدم داخل كود، بل بيئة تفاعلية كاملة (Interactive Computing Environment) تُشغَّل بداخلها NumPy وPandas وMatplotlib وPlotly وScikit-learn جميعًا. فهم بنيتها الداخلية ونموذج تنفيذها يفسّر سلوكيات كثيرة قد تبدو غريبة للمبتدئين، مثل بقاء متغير 'حيًا' في الذاكرة رغم حذف الخلية التي أنشأته.

الفصل الأول: التعريف بـ Jupyter

١.١ النشأة والتطور

بدأت القصة بمشروع IPython عام ٢٠٠١ على يد Fernando Pérez، كبديل تفاعلي محسَّن لمفسّر بايثون القياسي. لاحقًا، أضيفت له واجهة دفتر ملاحظات (Notebook) تجمع الكود والنص التوضيحي والمخرجات في مستند واحد قابل للتنفيذ. عام ٢٠١٤، انفصل الجزء الخاص بالدفاتر عن IPython ليصبح مشروعًا مستقلاً باسم Project Jupyter، بهدف صريح: دعم لغات برمجة متعددة وليس بايثون فقط — والاسم Jupyter نفسه اختصار مركّب من Julia وPython وR، أول ثلاث لغات استهدفها المشروع.

١.٢ المشكلة التي حلّها

قبل Jupyter، كانت دورة العمل النموذجية في تحليل البيانات هي: كتابة سكربت كامل، تشغيله من البداية للنهاية، ثم فحص المخرجات النصية فقط في الطرفية. لم تكن هناك طريقة طبيعية لدمج الكود مع الرسوم البيانية والنصوص التوضيحية والمعادلات الرياضية في مستند واحد قابل للتنفيذ التدريجي خطوة بخطوة، مع الاحتفاظ بحالة المتغيرات بين كل تنفيذ جزئي والآخر.

١.٣ الاستخدامات والشركات

- الاستكشاف التفاعلي للبيانات والتحليل الأولي السريع.
- التعليم الأكاديمي: دمج الشرح النظري بالكود القابل للتنفيذ مباشرة.
- النشر العلمي القابل لإعادة الإنتاج (Reproducible Research).
- النماذج الأولية السريعة (Prototyping) لنماذج التعلّم الآلي قبل تحويلها لكود إنتاجي.

تعتمد عليه كل شركات علم البيانات تقريبًا كأداة استكشاف يومية، وتستضيفه منصات سحابية كبرى مثل Google Colab وKaggle Notebooks وAmazon SageMaker كبيئة تشغيل موحّدة.

١.٤ المميزات والعيوب

الجانب	التفاصيل
مميزات	تنفيذ تدريجي تفاعلي، دمج كود ونص ورسوم في مستند واحد، معاينة فورية للنتائج، دعم لغات متعددة عبر Kernels مختلفة
عيوب	صعوبة التحكم بالإصدارات (Version Control) عبر Git بسبب صيغة الملف JSON المعقدة، سهولة الوقوع في أخطاء ترتيب التنفيذ (Hidden State)، أقل ملاءمة للكود الإنتاجي مقارنة بملفات .py عادية
حدود	ليست بديلاً عن هيكلة مشروع برمجي حقيقي بوحدات (Modules) واختبارات؛ الاستخدام المثالي استكشافي وتعليمي بالدرجة الأولى

١.٥ متى تستخدمه ومتى لا

القرار	الحالة
Jupyter Notebook	استكشاف بيانات جديدة، تجريب سريع، تعليم
ملفات .py عادية مع اختبارات	كود إنتاجي يعمل في نظام آلي مجدوَل (Scheduled Pipeline)
Jupyter Notebook (خاصة كملف مصدَّر لـ HTML)	مشاركة تحليل مع فريق يحتاج رؤية الخطوات والرسوم معًا
ملفات .py منظمة كحزمة، ليس دفاتر	مكتبة أو حزمة بايثون قابلة لإعادة الاستخدام

١.٦ أشهر البدائل والامتدادات

- JupyterLab: الواجهة الحديثة الأكثر تطورًا لـ Jupyter، بدعم علامات تبويب متعددة وتخطيط مرن.
- Google Colab: نسخة سحابية مجانية من Jupyter مع وصول لموارد GPU/TPU.
- VS Code Notebooks: دعم مدمج لملفات .ipynb داخل محرر VS Code مباشرة.
- Marimo: بديل حديث يعالج مشكلة "الحالة الخفية" عبر إعادة تنفيذ تفاعلية تلقائية تشبه جداول البيانات.

ملخص الفصل

- Jupyter تطوّر من IPython، وانفصل كمشروع مستقل لدعم لغات متعددة عبر Kernels.
- حلّ مشكلة غياب بيئة تفاعلية تدمج الكود والنص والرسوم في مستند واحد قابل لإعادة التنفيذ.
- الأنسب للاستكشاف والتعليم، وليس بديلاً عن هيكلة كود إنتاجي حقيقي.

الفصل الثاني: التثبيت والإعداد

```
pip install notebook          # Jupyter Notebook الكلاسيكي
pip install jupyterlab        # الواجهة الحديثة الموصى بها
conda install -c conda-forge jupyterlab
```

٢.١ التشغيل

```
jupyter lab           # تشغيل الواجهة الحديثة
jupyter notebook      # تشغيل الواجهة الكلاسيكية
```

٢.٢ إدارة Kernels متعددة البيئات

كل بيئة افتراضية (venv أو conda) تحتاج تسجيل Kernel منفصل خاص بها لتظهر كخيار داخل Jupyter، وإلا فإن Jupyter سيستخدم بيئة بايثون التي ثُبّت فيها هو نفسه فقط، بغض النظر عن أي بيئة افتراضية أخرى نشطة.

```
# داخل البيئة الافتراضية المطلوبة
pip install ipykernel
python -m ipykernel install --user --name=my_project_env --display-name 'Python
(my_project)'
```

٢.٣ تنظيم المشروع

```
project/
├── notebooks/
│   ├── 01_exploration.ipynb
│   ├── 02_cleaning.ipynb
│   └── 03_modeling.ipynb
├── src/                  # كود قابل لإعادة الاستخدام يُستورد داخل الدفاتر
└── data/
```

ترقيم الدفاتر (٠١_, ٠٢_, ٠٣_) هو اصطلاح شائع جدًا في المشاريع الاحترافية للدلالة على ترتيب سير العمل المنطقي عبر عدة دفاتر منفصلة الغرض.

أفضل الممارسات

- سجّل Kernel منفصل لكل بيئة افتراضية عبر ipykernel بدل الاعتماد على بيئة Jupyter الافتراضية.
- رقّم أسماء الدفاتر حسب ترتيب سير العمل المنطقي (استكشاف، تنظيف، نمذجة).
- انقل أي دالة تتكرر استخدامها عبر عدة دفاتر إلى وحدة .py في src/ واستوردها بدل تكرار الكود.

الفصل الثالث: البنية الداخلية ونموذج التنفيذ

٣.١ العمارة: Frontend وKernel منفصلان

التصميم المحوري لـ Jupyter هو الفصل التام بين الواجهة (Frontend)، أي المتصفح الذي تكتب فيه الكود وترى المخرجات) وMحرك التنفيذ (Kernel، العملية المستقلة التي تُنفّذ الكود فعليًا). يتواصل الاثنان عبر بروتوكول رسائل موحّد (Jupyter Messaging Protocol) مبني فوق ZeroMQ، وهذا الفصل هو ما يسمح بوجود Kernels للغات مختلفة كليًا (بايثون، Julia، R، حتى Bash) خلف نفس واجهة الدفتر تمامًا.

الدور	المكوّن
عرض الخلايا، تحرير الكود والنص، عرض المخرجات	Frontend (المتصفح)
عملية منفصلة تُنفّذ الكود فعليًا وتحتفظ بحالة المتغيرات	Kernel
الوسيط الذي يدير الاتصال بين المتصفح وKernel، ويخدم ملفات ipynb.	Jupyter Server
ملف JSON يخزّن الخلايا، الكود، المخرجات، وMetadata	Notebook Document (.ipynb)

٣.٢ Execution Model: حالة مشتركة عبر الخلايا

المفهوم الأهم لفهمه بعمق: كل الخلايا (Cells) في دفتر واحد تتشارك نفس مساحة الذاكرة (Namespace) داخل Kernel واحد. الترتيب البصري للخلايا على الشاشة لا يحدد ترتيب التنفيذ الفعلي؛ ما يحدده هو ترتيب تشغيلك الفعلي لها، والذي يظهر كرقم [١]، [٢]، [٣] بجانب كل خلية بعد تنفيذها. يمكنك تنفيذ الخلية الخامسة، ثم العودة للثانية وتنفيذها، وستستخدم أي متغيرات عرّفتها الخامسة إن كانت موجودة بالفعل في الذاكرة — وهو مصدر أخطاء شهير جدًا يُعرف بـ "الحالة الخفية" (Hidden State).

```
# الخلية [1]
x = 10

# الخلية [2] (من الخلية [1] x نُفِّذت لاحقًا، لكنها تعتمد على)
print(x * 2)   # يعمل بشرط تنفيذ [1] فعليًا قبلها، بصرف النظر عن ترتيبهما البصري
```

٣.٣ لماذا هذا يسبب مشاكل: مشكلة الحالة الخفية

لو حذفت الخلية [١] بعد تنفيذها، يبقى المتغير x حيًا في ذاكرة Kernel رغم اختفاء الكود الذي أنشأه من الدفتر المرئي! هذا يعني أن الدفتر الذي "يعمل" أمامك الآن قد لا يعمل إطلاقًا إذا أعدت تشغيله من الصفر (Restart & Run All)، لأن بعض الخلايا تعتمد على حالة لم تعد موجودة في الكود المرئي. هذا هو السبب الأول وراء توصية "Restart & Run All" الدورية أثناء التطوير كفحص سلامة أساسي.

٣.٤ أنواع الخلايا

الاستخدام	نوع الخلية
كود قابل للتنفيذ باللغة المرتبطة بالـ Kernel النشط	Code

نوع الخلية	الاستخدام
Markdown	نص توضيحي، عناوين، معادلات LaTeX، صور
Raw	نص خام لا يُعالَج أو يُنفَّذ، يُستخدم غالبًا مع أدوات تحويل مثل nbconvert

ملخص الفصل

- العمارة تفصل بين Frontend (المتصفح) وKernel (محرك التنفيذ)، متصلان عبر بروتوكول رسائل موحّد.
  - كل الخلايا تتشارك حالة واحدة في ذاكرة Kernel؛ ترتيب التنفيذ الفعلي هو ما يهم، لا الترتيب البصري.
  - "الحالة الخفية" مشكلة جوهرية ناتجة عن هذا التصميم، ويُنصح بفحصها دوريًا عبر Restart & Run All.

الفصل الرابع: المفاهيم الأساسية

٤.١ دورة حياة Kernel

- Start: بدء عملية Kernel جديدة عند فتح دفتر أو إعادة تشغيله.
- Busy/Idle: حالة Kernel أثناء تنفيذ خلية (Busy) أو انتظار أمر جديد (Idle)، تظهر كنقطة ملوّنة في الواجهة.
- Interrupt: إيقاف تنفيذ خلية عالقة (كحلقة لا نهائية) دون إغلاق Kernel بأكمله.
- Restart: إعادة تشغيل Kernel بالكامل، مما يمسح كل المتغيرات ويبدأ ذاكرة نظيفة تمامًا.
- Shutdown: إيقاف Kernel كليًا عند إغلاق الدفتر.

٤.٢ Markdown داخل الدفاتر

```
# عنوان رئيسي
## عنوان فرعي
**نص عريض** و *نص مائل*
- عنصر قائمة
$E = mc^2$   <!-- معادلة رياضية عبر LaTeX -->
```

٤.٣ Magic Commands

الأوامر السحرية (Magic Commands) هي أوامر خاصة بـ IPython تبدأ بـ % (لسطر واحد) أو %% (للخلية كاملة)، وهي ليست جزءًا من لغة بايثون نفسها بل ميزة توفّرها بيئة IPython/Jupyter تحديدًا.

الأمر	الاستخدام
timeit%	قياس زمن تنفيذ تعبير واحد بدقة إحصائية
time%%	قياس زمن تنفيذ الخلية كاملة (تشغيل واحد فقط)
matplotlib inline%	عرض رسوم Matplotlib كصور مضمّنة في الدفتر
load_ext%	تحميل امتداد IPython إضافي (مثل line_profiler)
who / %whos%	عرض كل المتغيرات الحالية في ذاكرة Kernel
writefile%%	كتابة محتوى الخلية كاملاً إلى ملف على القرص
command!	تنفيذ أمر Shell مباشرة (مثل !pip install)

```
%%time
result = sum(i**2 for i in range(1_000_000))

!pip list | grep numpy
```

٤.٤ Widgets للتفاعلية

```
from ipywidgets import interact

@interact(x=(0, 10))
```

```
def show_square(x=5):
    print(x ** 2)
```

interact تحوّل معاملات أي دالة إلى عناصر تحكم تفاعلية (شريط تمرير، قائمة منسدلة) تلقائيًا حسب نوع القيمة الافتراضية، وهي أداة ممتازة للاستكشاف التفاعلي السريع دون بناء واجهة Dash كاملة.

الأخطاء الشائعة

- الاعتماد على ترتيب الخلايا البصري بدل ترتيب التنفيذ الفعلي عند تشخيص خطأ.
- نسيان أن timeit% يقيس تعبيرًا واحدًا، بينما time%% يقيس الخلية كاملة مرة واحدة فقط بلا تكرار إحصائي.
- استخدام !pip install داخل دفتر يعمل على Kernel مختلف عن بيئة pip الافتراضية في الطرفية، مما يثبّت الحزمة في المكان الخطأ.

تمارين عملية

١. أنشئ دفترًا بثلاث خلايا: Markdown بعنوان، كود يُعرّف دالة، وكود يستدعيها. نفّذها بترتيب معكوس وراقب النتيجة.

٢. استخدم timeit% لمقارنة زمن list comprehension مقابل حلقة for عادية لإنشاء قائمة من ١٠٠٠٠٠ عنصر.

٣. استخدم interact@ لبناء أداة تفاعلية تعرض رسم sin(x * freq) مع شريط تمرير للتردد freq.

حلول مقترحة (تلميحات)

١. لاحظ أن استدعاء الدالة يعمل فقط إذا نُفِّذت خلية التعريف فعليًا قبله بصرف النظر عن ترتيبها البصري.

٢. timeit [i**2 for i in range(100000)]% مقابل time%% مع حلقة for يدوية.

٣. @interact(freq=(1, 10)) def plot_wave(freq=1): ثم ارسم عبر Matplotlib داخل الدالة.

الفصل الخامس: الامتدادات وأدوات التصحيح والقياس

٥.١ التصحيح (Debugging) داخل Jupyter

```
%debug    # يفتح مصحّح تفاعلي (pdb) عند آخر استثناء حدث في الخلية السابقة

# أو تفعيل التصحيح التلقائي عند أي استثناء لاحق
%pdb on
```

debug% بعد حدوث خطأ يفتح جلسة تصحيح تفاعلية تضعك مباشرة عند نقطة الاستثناء، حيث يمكنك فحص قيم المتغيرات في تلك اللحظة بالضبط دون إعادة تشغيل الكود من البداية.

٥.٢ القياس والملفات الشخصية (Profiling)

```
%load_ext line_profiler
%lprun -f my_function my_function(data)

%load_ext memory_profiler
%memit my_function(data)
```

٥.٣ اختصارات لوحة المفاتيح الأساسية

الوظيفة	الاختصار
تنفيذ الخلية الحالية والانتقال للتالية	Shift + Enter
تنفيذ الخلية الحالية والبقاء فيها	Ctrl/Cmd + Enter
إضافة خلية أعلى (A) أو أسفل (B) الخلية الحالية	Esc ثم A / B
حذف الخلية الحالية (ضغط D مرتين)	Esc ثم D D
تحويل الخلية إلى Markdown (M) أو Code (Y)	Esc ثم M / Y
إعادة تشغيل Kernel (Restart)	Esc ثم ٠ ٠

٥.٤ الامتدادات الشائعة (Extensions)

- jupyterlab-git: تكامل مباشر مع Git داخل واجهة JupyterLab.
- nbdime: أداة مقارنة (Diff) ودمج (Merge) ذكية لملفات .ipynb، تتعامل مع بنية الخلايا لا نص JSON الخام.
- jupytext: يزامن كل دفتر .ipynb مع نسخة .py مقابلة، مما يجعل تتبع Git أوضح بكثير.
- nbconvert: تحويل دفاتر إلى صيغ أخرى (HTML, PDF, Python script) من سطر الأوامر.

```
jupyter nbconvert --to html notebook.ipynb
jupyter nbconvert --to script notebook.ipynb   # استخراج الكود فقط كملف .py
```

أفضل الممارسات

- استخدم debug% فور حدوث استثناء بدل إعادة تشغيل الخلية بتعديلات تخمينية متكررة.

- ثبّت nbdime إن كنت تستخدم Git مع دفاتر Jupyter بجدية، لتفادي فروقات JSON غير المقروءة.
- احفظ عادة الضغط على 'Restart & Run All' دوريًا للتأكد من خلو الدفتر من الحالة الخفية.

الفصل السادس: التنظيم وأسلوب العمل الاحترافي

٦.١ هيكلة دفتر واحد جيدة التنظيم

- قسم استيراد المكتبات في أعلى الدفتر دائمًا في خلية واحدة أولى.
- استخدم خلايا Markdown كعناوين لفصل مراحل التحليل المنطقية (تحميل، تنظيف، استكشاف، نمذجة).
- ضع الثوابت والإعدادات (مسارات الملفات، معاملات النموذج) في خلية واحدة قريبة من البداية لسهولة التعديل.
- تجنّب خلايا ضخمة جدًا تقوم بعشر مهام مختلفة؛ خلية واحدة لكل خطوة منطقية واضحة.

٦.٢ من الاستكشاف إلى الإنتاج

النمط الاحترافي المتّبع غالبًا هو: استكشاف حر وسريع داخل الدفاتر، ثم بمجرد استقرار منطق معيّن (دالة تنظيف بيانات، دالة رسم متكررة)، يُنقَل ذلك المنطق فورًا إلى ملف .py داخل src/ ويُستورَد من هناك، بدل تركه مكررًا عبر عدة دفاتر.

```
# داخل دفتر الاستكشاف، بعد استقرار الدالة
from src.preprocessing import clean_customer_data

df_clean = clean_customer_data(df_raw)   # الدفتر يستدعي فقط، لا يحتوي المنطق نفسه
```

٦.٣ عرض تقديمي (Presentation Mode)

```
jupyter nbconvert notebook.ipynb --to slides --post serve
```

nbconvert قادر على تحويل دفتر كامل إلى عرض شرائح تفاعلي (مبني على reveal.js) مباشرة من سطر الأوامر، بالاعتماد على Metadata نوع الشريحة (Slide Type) المُحدَّد لكل خلية عبر إعدادات العرض في الواجهة.

٦.٤ التصدير والمشاركة

الأمر	الصيغة
jupyter nbconvert --to html notebook.ipynb	HTML ثابت للمشاركة السريعة
jupyter nbconvert --to pdf notebook.ipynb	PDF (يتطلب LaTeX مثبّتًا)
jupyter nbconvert --to script notebook.ipynb	ملف بايثون خام
jupyter nbconvert --to slides notebook.ipynb	عرض شرائح تفاعلي

الأخطاء الشائعة

- ترك خلايا استكشاف تجريبية فوضوية في الدفتر النهائي المُشارَك بدل تنظيفه أولاً (Clean & Restart & Run All قبل المشاركة).
- تكرار نفس دالة التنظيف أو الرسم يدويًا عبر عدة دفاتر بدل استخراجها لوحدة .py مشتركة.

الأسئلة الشائعة

س: هل يجب أن يبدأ كل مشروع تعلّم آلي بدفتر Jupyter؟

ج: غالبًا نعم للاستكشاف الأولي؛ لكن مع نضج المشروع، يُفضَّل انتقال المنطق المستقر تدريجيًا إلى كود .py منظم واختبارات، مع بقاء الدفاتر للاستكشاف والتقارير النهائية فقط.

تمارين عملية

١. نظّم دفترًا فوضويًا افتراضيًا (استيرادات متناثرة، خلايا غير مرتبة) عبر إعادة ترتيبه بخلايا Markdown فاصلة بين المراحل.

٢. صدّر أي دفتر لديك إلى HTML وإلى ملف .py عبر nbconvert وقارن الناتجين.

٣. استخرج دالة رسم متكررة من دفتر إلى ملف src/viz.py واستوردها بدلاً من تكرارها.

حلول مقترحة (تلميحات)

١. اجمع كل import في خلية أولى، وأضف ## عناوين Markdown قبل كل مرحلة (تحميل، تنظيف، تحليل).

٢. jupyter nbconvert --to html notebook.ipynb && jupyter nbconvert --to script notebook.ipynb

٣. انقل جسم الدالة إلى src/viz.py مع from src.viz import my_plot_function في الدفتر.

مراجع وروابط رسمية

- التوثيق الرسمي لـ Jupyter: https://docs.jupyter.org/
- توثيق JupyterLab: https://jupyterlab.readthedocs.io/
- الكود المصدري: https://github.com/jupyter/notebook

خاتمة الجزء الأول من Jupyter

غطّينا نشأة Jupyter وعمارته القائمة على فصل Frontend عن Kernel، ونموذج التنفيذ التشاركي بين الخلايا ومشكلة الحالة الخفية، إضافة إلى الأوامر السحرية وأدوات التصحيح والتنظيم. الجزء الثاني والأخير من هذا الكتاب سيغطي الفصول السابع إلى الحادي عشر لـ Jupyter، ثم فصل المقارنات الشامل بين كل المكتبات الست وWorkflow احترافي متكامل يربطها جميعًا.